

FINAL PROJECT DOCUMENTATION

AI Stock Market Prediction System

Generated on: 12 May 2026, 02:58 AM

Student: Yash Bansal

Project Folder: Desktop / FINAL PROJECT

1. Project Overview

This project is a Streamlit-based AI Stock Market Prediction System. It was converted from a Colab-style project into a complete local desktop application with login, signup, user dashboard, watchlist, stock prediction, separate graph outputs, GitHub version control, and deployment readiness.

2. Final Features Added

Feature	Description
Login and Signup	Users can create an account and log in before accessing the dashboard.
Secure Password Storage	Passwords are stored using salted PBKDF2 hashing in local users.json.
User Dashboard	Authenticated users get a dashboard with account information and watchlist management.
Watchlist	Users can save selected stocks and view latest stock records.
Stock Prediction	Random Forest models predict next closing price and UP/DOWN direction.
Separate Graph Outputs	Candlestick, close trend, moving averages, volume, daily return, volatility, and prediction graph are shown separately.
GitHub Push	The latest project was pushed to GitHub repository Bansal-yash08/stock-market-project-.
Live Sharing	A Cloudflare tunnel was created for temporary public access while the laptop is running.

3. Graphs Shown Separately in Output

1. Candlestick Price Action
2. Closing Price Trend
3. Moving Average Comparison
4. Trading Volume
5. Daily Return
6. Volatility
7. Actual vs Predicted Next Close

4. Technologies Used

Python 3.12, Streamlit, Pandas, NumPy, Plotly, Scikit-learn, yfinance, Git, GitHub, VS Code, and Cloudflare Tunnel.

5. Important Links

GitHub Repository: <https://github.com/Bansal-yash08/stock-market-project>

Local App URL: <http://localhost:8501>

Temporary Live Tunnel URL: <https://wallace-designed-given-diet.trycloudflare.com>

Note: The tunnel link works only while the laptop and tunnel process are running.

6. How to Run the Project

1. Open the FINAL PROJECT folder.
2. Double-click run_app.bat.
3. Open <http://localhost:8501> in the browser.
4. Create an account using the signup page.
5. Login and use the user dashboard and stock prediction page.

7. Project Files

File	Purpose
app.py	Main Streamlit application with authentication, dashboard, models, and graphs.
requirements.txt	Python package list for setup and deployment.
nifty500_stocks.csv	Historical stock dataset.
run_app.bat	Windows double-click runner.
run_app.ps1	PowerShell runner.
README.md	Project instructions and deployment notes.
PROJECT_DESCRIPTION.txt	Brief Notepad project explanation.
project documentation clg.docx	College project documentation updated for final app.

8. Project Description Text

■FINAL PROJECT - AI Stock Market Prediction System

Project Description:

This project is a local Streamlit web application for predicting stock market movement using historical Nifty 500 stock data. It was

Main Features:

1. Login and Signup system
 - Users can create an account using name, email, and password.
 - Passwords are saved using salted hashing, not plain text.
 - Users can log in and log out.
2. User Dashboard
 - After login, the user gets a dashboard page.
 - The dashboard includes a personal watchlist.
 - The watchlist is saved locally in users.json after signup/use.
3. Stock Prediction Dashboard
 - User can select a stock from the Nifty 500 dataset.
 - The app loads historical stock data from nifty500_stocks.csv.
 - It can also add recent Yahoo Finance data if internet data is available.
 - It creates technical features such as daily return, volatility, moving averages, EMA, and lag values.
 - It trains Random Forest models for next close price prediction and up/down direction prediction.
 - It displays prediction metrics, signal, candlestick chart, data table, features table, and model scores.

Files Included:

- app.py: Main Streamlit application with login, signup, user dashboard, and prediction dashboard.
- requirements.txt: Python libraries required to run the project.
- run_app.bat: Double-click runner for Windows.
- run_app.ps1: PowerShell runner.
- README.md: Basic project instructions.
- nifty500_stocks.csv: Historical stock dataset used by the app.

How to Run:

1. Open the FINAL PROJECT folder.
2. Double-click run_app.bat.
3. The app will start in the browser at <http://localhost:8501>.
4. Create a new account on the signup page.
5. After signup/login, use the dashboard and stock prediction page.

Important Note:

This project is for educational/demo purposes only. The predictions should not be used as real financial or trading advice.

9. README Content

AI Stock Market Prediction System

This is a cleaned local version of the Colab stock prediction project.
It includes login, signup, a user dashboard, watchlist saving, and the stock prediction dashboard.

Run

```
```powershell
pip install -r requirements.txt
streamlit run app.py
```
```

The app expects `nifty500\_stocks.csv` to be in the same folder as `app.py`.
User accounts are stored locally in `users.json` after signup. Passwords are stored with a salted hash, not plain text.

On this Windows machine, Python was installed at:

```
```text
C:\Users\YASH BANSAL\AppData\Local\Programs\Python\Python312\python.exe
```
```

You can also double-click `run\_app.bat`, or run:

```
```powershell
powershell -ExecutionPolicy Bypass -File .\run_app.ps1
```
```

Deploy

This project is ready for Streamlit Community Cloud.

1. Push this folder to GitHub.
2. Open <https://share.streamlit.io/>
3. Click "New app".
4. Select the GitHub repository.
5. Set the main file path to `app.py`.
6. Deploy.

Note: Streamlit Community Cloud storage can reset when the app restarts. For production user accounts, connect a real database instead.

10. Requirements

```
streamlit  
pandas  
numpy  
plotly  
scikit-learn  
yfinance
```

11. Full Source Code: app.py

```
import hashlib
import hmac
import json
import secrets
import warnings
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st

warnings.filterwarnings("ignore")

DATA_PATH = "nifty500_stocks.csv"
USERS_PATH = Path("users.json")

st.set_page_config(page_title="AI Stock Market Prediction System", layout="wide")

def load_users() -> dict:
    if not USERS_PATH.exists():
        return {}
    try:
        return json.loads(USERS_PATH.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return {}

def save_users(users: dict) -> None:
    USERS_PATH.write_text(json.dumps(users, indent=2), encoding="utf-8")

def hash_password(password: str, salt: str | None = None) -> str:
    salt = salt or secrets.token_hex(16)
    password_hash = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), bytes.fromhex(salt), 120000)
    return f"{salt}${password_hash.hex()}"

def verify_password(password: str, saved_password: str) -> bool:
    try:
        salt, stored_hash = saved_password.split("$", 1)
    except ValueError:
        return False
    return hmac.compare_digest(hash_password(password, salt), f"{salt}${stored_hash}")

def normalize_email(email: str) -> str:
    return email.strip().lower()

def initialize_session() -> None:
    st.session_state.setdefault("authenticated", False)
    st.session_state.setdefault("user_email", "")
    st.session_state.setdefault("user_name", "")

def show_auth_page() -> None:
    st.title("AI Stock Market Prediction System")
    st.subheader("Login or create your account")

    login_tab, signup_tab = st.tabs(["Login", "Sign up"])

    with login_tab:
        with st.form("login_form"):
            email = st.text_input("Email", key="login_email")
            password = st.text_input("Password", type="password", key="login_password")
            submitted = st.form_submit_button("Login", use_container_width=True)

            if submitted:
                users = load_users()
                email_key = normalize_email(email)
                user = users.get(email_key)
                if user and verify_password(password, user["password"]):
                    st.session_state.authenticated = True
                    st.session_state.user_email = email_key
                    st.session_state.user_name = user["name"]
```

```

        st.success("Login successful.")
        st.rerun()
    else:
        st.error("Invalid email or password.")

with signup_tab:
    with st.form("signup_form"):
        name = st.text_input("Full name")
        email = st.text_input("Email", key="signup_email")
        password = st.text_input("Password", type="password", key="signup_password")
        confirm_password = st.text_input("Confirm password", type="password")
        submitted = st.form_submit_button("Create account", use_container_width=True)

    if submitted:
        users = load_users()
        email_key = normalize_email(email)
        if not name.strip():
            st.error("Please enter your name.")
        elif "@" not in email_key or "." not in email_key:
            st.error("Please enter a valid email address.")
        elif len(password) < 6:
            st.error("Password must be at least 6 characters.")
        elif password != confirm_password:
            st.error("Passwords do not match.")
        elif email_key in users:
            st.error("An account already exists for this email.")
        else:
            users[email_key] = {
                "name": name.strip(),
                "password": hash_password(password),
                "created_at": datetime.now().isoformat(timespec="seconds"),
                "watchlist": ["RELIANCE"],
            }
            save_users(users)
            st.session_state.authenticated = True
            st.session_state.user_email = email_key
            st.session_state.user_name = name.strip()
            st.success("Account created.")
            st.rerun()

def logout() -> None:
    st.session_state.authenticated = False
    st.session_state.user_email = ""
    st.session_state.user_name = ""
    st.rerun()

@st.cache_data(show_spinner=False)
def load_dataset(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    df = df.drop_duplicates()
    df = df.drop(columns=["Adj Close"], errors="ignore")
    df = df.rename(columns={"Symbol": "Stocks_Name"})
    df["Date"] = pd.to_datetime(df["Date"], format="%d-%m-%Y", errors="coerce")

    price_columns = ["Open", "High", "Low", "Close", "Volume"]
    for column in price_columns:
        df[column] = pd.to_numeric(df[column], errors="coerce")

    df = df.dropna(subset=["Date", "Stocks_Name", "Open", "High", "Low", "Close", "Volume"])
    df["Stocks_Name"] = df["Stocks_Name"].astype(str).str.upper().str.strip()
    return df.sort_values(["Stocks_Name", "Date"]).reset_index(drop=True)

def add_features(df: pd.DataFrame) -> pd.DataFrame:
    featured = df.copy()
    featured["Daily_Return"] = featured["Close"].pct_change()
    featured["Volatility"] = featured["Daily_Return"].rolling(20, min_periods=5).std()
    featured["MA_10"] = featured["Close"].rolling(10, min_periods=3).mean()
    featured["MA_20"] = featured["Close"].rolling(20, min_periods=5).mean()
    featured["EMA_10"] = featured["Close"].ewm(span=10, adjust=False).mean()
    featured["EMA_20"] = featured["Close"].ewm(span=20, adjust=False).mean()
    featured["Lag1"] = featured["Close"].shift(1)
    featured["Lag2"] = featured["Close"].shift(2)
    featured["Target_Close"] = featured["Close"].shift(-1)
    featured["Target_Direction"] = (featured["Target_Close"] > featured["Close"]).astype(int)
    return featured.dropna().reset_index(drop=True)

@st.cache_resource(show_spinner=False)
def train_model(featured: pd.DataFrame):
    from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
    from sklearn.metrics import accuracy_score, mean_absolute_error, r2_score

```

```

from sklearn.model_selection import train_test_split

features = [
    "Open",
    "High",
    "Low",
    "Close",
    "Volume",
    "Daily_Return",
    "Volatility",
    "MA_10",
    "MA_20",
    "EMA_10",
    "EMA_20",
    "Lag1",
    "Lag2",
]
x = featured[features]
y_price = featured["Target_Close"]
y_direction = featured["Target_Direction"]

split_kwargs = {"test_size": 0.2, "shuffle": False}
x_train, x_test, y_price_train, y_price_test = train_test_split(x, y_price, **split_kwargs)
_, _, y_direction_train, y_direction_test = train_test_split(x, y_direction, **split_kwargs)

regressor = RandomForestRegressor(n_estimators=180, random_state=42, min_samples_leaf=2)
classifier = RandomForestClassifier(n_estimators=180, random_state=42, min_samples_leaf=2)
regressor.fit(x_train, y_price_train)
classifier.fit(x_train, y_direction_train)

price_predictions = regressor.predict(x_test)
direction_predictions = classifier.predict(x_test)
metrics = {
    "mae": mean_absolute_error(y_price_test, price_predictions),
    "r2": r2_score(y_price_test, price_predictions),
    "accuracy": accuracy_score(y_direction_test, direction_predictions),
}

return regressor, classifier, features, metrics

def latest_yahoo_data(symbol: str) -> pd.DataFrame:
    try:
        import yfinance as yf

        data = yf.download(symbol, period="5d", interval="15m", progress=False, auto_adjust=False)
        if data.empty:
            return pd.DataFrame()
        if isinstance(data.columns, pd.MultiIndex):
            data.columns = data.columns.get_level_values(0)
        data = data.reset_index().rename(columns={"Datetime": "Date"})
        data["Date"] = pd.to_datetime(data["Date"]).dt.tz_localize(None)
        return data[["Date", "Open", "High", "Low", "Close", "Volume"]].dropna()
    except Exception:
        return pd.DataFrame()

def candlestick_chart(df: pd.DataFrame, title: str) -> go.Figure:
    fig = go.Figure(
        data=[
            go.Candlestick(
                x=df["Date"],
                open=df["Open"],
                high=df["High"],
                low=df["Low"],
                close=df["Close"],
                name="Price",
            )
        ]
    )
    fig.update_layout(title=title, xaxis_rangeslider_visible=False, yaxis_title="Price")
    return fig

def line_chart(df: pd.DataFrame, columns: list[str], title: str, y_title: str) -> go.Figure:
    fig = go.Figure()
    for column in columns:
        fig.add_trace(go.Scatter(x=df["Date"], y=df[column], mode="lines", name=column))
    fig.update_layout(title=title, xaxis_title="Date", yaxis_title=y_title, hovermode="x unified")
    return fig

def volume_chart(df: pd.DataFrame, title: str) -> go.Figure:
    fig = go.Figure(data=[go.Bar(x=df["Date"], y=df["Volume"], name="Volume")])

```

```

fig.update_layout(title=title, xaxis_title="Date", yaxis_title="Volume")
return fig

def prediction_chart(df: pd.DataFrame, predicted_close: float, title: str) -> go.Figure:
    recent = df.tail(120).copy()
    next_date = recent["Date"].iloc[-1] + pd.Timedelta(days=1)
    fig = go.Figure()
    fig.add_trace(go.Scatter(x=recent["Date"], y=recent["Close"], mode="lines", name="Actual Close"))
    fig.add_trace(
        go.Scatter(
            x=[next_date],
            y=[predicted_close],
            mode="markers+text",
            name="Predicted Next Close",
            text=["Prediction"],
            textposition="top center",
            marker={"size": 12},
        )
    )
    fig.update_layout(title=title, xaxis_title="Date", yaxis_title="Close Price", hovermode="x unified")
    return fig

initialize_session()

if not st.session_state.authenticated:
    show_auth_page()
    st.stop()

with st.sidebar:
    st.header("User Dashboard")
    st.write(f"Welcome, {st.session_state.user_name}")
    st.write(st.session_state.user_email)
    st.divider()
    dashboard_page = st.radio("Menu", ["Stock Prediction", "My Dashboard"], label_visibility="collapsed")
    if st.button("Logout", use_container_width=True):
        logout()

st.title("AI Stock Market Prediction System")
st.caption(f"Signed in as {st.session_state.user_name}")

try:
    market_df = load_dataset(DATA_PATH)
except FileNotFoundError:
    st.error("nifty500_stocks.csv was not found beside app.py.")
    st.stop()

stock_names = sorted(market_df["Stocks_Name"].unique())

if dashboard_page == "My Dashboard":
    users = load_users()
    current_user = users.get(st.session_state.user_email, {})
    watchlist = current_user.get("watchlist", ["RELIANCE"])

    st.subheader("User Dashboard")
    dash_cols = st.columns(3)
    dash_cols[0].metric("Account", current_user.get("name", st.session_state.user_name))
    dash_cols[1].metric("Watchlist stocks", len(watchlist))
    dash_cols[2].metric("Available stocks", f"{len(stock_names):,}")

    selected_watchlist = st.multiselect("Watchlist", stock_names, default=[stock for stock in watchlist if stock in stock_names])
    if st.button("Save watchlist"):
        users[st.session_state.user_email]["watchlist"] = selected_watchlist
        save_users(users)
        st.success("Watchlist saved.")

    if selected_watchlist:
        latest_rows = (
            market_df[market_df["Stocks_Name"].isin(selected_watchlist)]
            .sort_values("Date")
            .groupby("Stocks_Name")
            .tail(1)[["Stocks_Name", "Date", "Open", "High", "Low", "Close", "Volume"]]
            .sort_values("Stocks_Name")
        )
        st.dataframe(latest_rows, use_container_width=True)
    else:
        st.info("Add stocks to your watchlist to see them here.")

    st.stop()

left, right = st.columns([1, 2])
with left:
    selected_stock = st.selectbox("Stock", stock_names, index=stock_names.index("RELIANCE") if "RELIANCE" in stock_names else 0)

```

```

        use_live_data = st.toggle("Add recent Yahoo Finance data", value=True)
with right:
    st.metric("Dataset rows", f"{len(market_df):,}")
    st.metric("Available stocks", f"{len(stock_names):,}")

stock_df = market_df[market_df["Stocks_Name"] == selected_stock][["Date", "Open", "High", "Low", "Close", "Volume"]].copy()

live_df = pd.DataFrame()
if use_live_data:
    live_df = latest_yahoo_data(f"{selected_stock}.NS")
    if not live_df.empty:
        stock_df = pd.concat([stock_df[stock_df["Date"] < live_df["Date"].min()], live_df], ignore_index=True)

stock_df = stock_df.sort_values("Date").reset_index(drop=True)
featured_df = add_features(stock_df)

if len(featured_df) < 80:
    st.warning("Not enough rows are available to train a reliable model for this stock.")
    st.stop()

regressor, classifier, feature_columns, model_metrics = train_model(featured_df)
latest_row = featured_df[feature_columns].tail(1)
next_close = float(regressor.predict(latest_row)[0])
direction = int(classifier.predict(latest_row)[0])
current_close = float(featured_df["Close"].iloc[-1])
change = next_close - current_close
change_pct = (change / current_close) * 100

metric_cols = st.columns(4)
metric_cols[0].metric("Latest close", f"{current_close:.2f}")
metric_cols[1].metric("Predicted next close", f"{next_close:.2f}", f"{change_pct:.2f}%")
metric_cols[2].metric("Signal", "UP" if direction == 1 else "DOWN")
metric_cols[3].metric("Model accuracy", f"{model_metrics['accuracy'] * 100:.1f}%")

recent_stock = stock_df.tail(250)
recent_features = featured_df.tail(250)

st.subheader("Separate Graph Outputs")

st.markdown("#### 1. Candlestick Price Action")
st.plotly_chart(candlestick_chart(recent_stock, f"{selected_stock} Candlestick Price Action"), use_container_width=True)

st.markdown("#### 2. Closing Price Trend")
st.plotly_chart(line_chart(recent_features, ["Close"], f"{selected_stock} Closing Price Trend", "Close Price"), use_container_width=True)

st.markdown("#### 3. Moving Average Comparison")
st.plotly_chart(
    line_chart(recent_features, ["Close", "MA_10", "MA_20", "EMA_10", "EMA_20"], f"{selected_stock} Moving Averages", "Price"),
    use_container_width=True,
)

st.markdown("#### 4. Trading Volume")
st.plotly_chart(volume_chart(recent_stock, f"{selected_stock} Trading Volume"), use_container_width=True)

st.markdown("#### 5. Daily Return")
st.plotly_chart(line_chart(recent_features, ["Daily_Return"], f"{selected_stock} Daily Return", "Daily Return"), use_container_width=True)

st.markdown("#### 6. Volatility")
st.plotly_chart(line_chart(recent_features, ["Volatility"], f"{selected_stock} Rolling Volatility", "Volatility"), use_container_width=True)

st.markdown("#### 7. Actual vs Predicted Next Close")
st.plotly_chart(prediction_chart(featured_df, next_close, f"{selected_stock} Prediction Output"), use_container_width=True)

tab_data, tab_features, tab_model = st.tabs(["Data", "Features", "Model"])
with tab_data:
    st.dataframe(stock_df.tail(50), use_container_width=True)

with tab_features:
    st.dataframe(featured_df.tail(50), use_container_width=True)

with tab_model:
    st.write(
        {
            "Mean absolute error": round(model_metrics["mae"], 3),
            "R2 score": round(model_metrics["r2"], 3),
            "Direction accuracy": round(model_metrics["accuracy"], 3),
        }
    )
    st.caption("Educational project only. Do not use this dashboard as financial advice.")

```